

AegisX – AI-Powered Cybersecurity Assessment Platform

Software Requirements Specification (SRS) Document

Document Standard: IEEE Std 830-1998 Compliant

Version: 1.0.0

Status: Baseline Specification

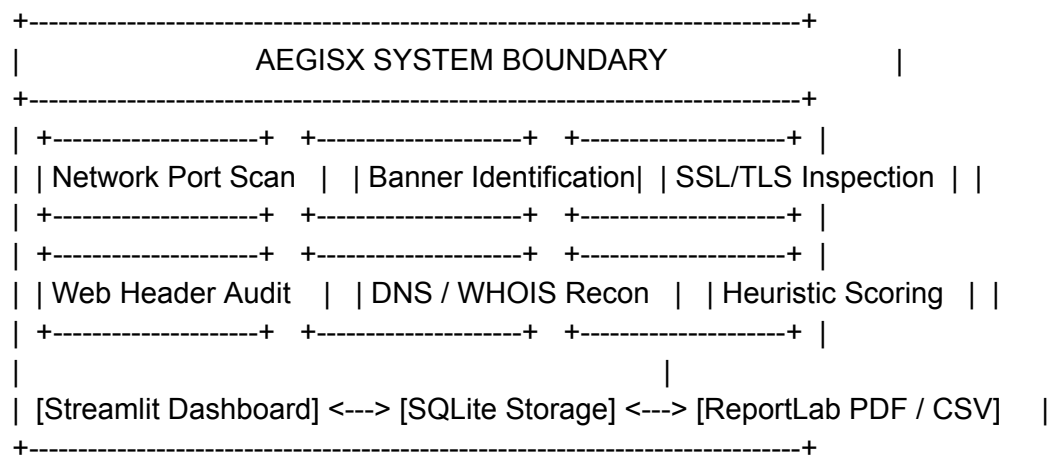
1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document details the complete functional, performance, architectural, and interface specifications for **AegisX – AI-Powered Cybersecurity Assessment Platform**. Designed following IEEE 830 standards, this document serves as the primary technical blueprint for software engineering, system integration, automated testing, and portfolio verification.

1.2 Scope

AegisX is an open-source, lightweight security auditing engine built in Python that consolidates network port scanning, service banner identification, HTTP security header evaluation, SSL/TLS certificate inspection, and DNS/WHOIS reconnaissance into a single web dashboard. The system aggregates scan findings into a heuristic risk scoring model (\$0.0 - 10.0\$) and generates downloadable PDF executive summaries and CSV dataset exports.



1.3 Intended Audience

- **Software Engineers & Security Developers:** For technical implementation and module development.
- **Academic Evaluators & Hackathon Judges:** For assessing architectural design, security rigor, and code quality.

- **Technical Recruiters:** For evaluating full-stack software development and network security competencies.

1.4 Definitions, Acronyms, and Abbreviations

Term / Acronym	Full Form / Definition
API	Application Programming Interface
BANNER	Text string sent by a network service indicating server software and version
CSP	Content Security Policy (HTTP Security Header)
CVE	Common Vulnerabilities and Exposures
HSTS	HTTP Strict Transport Security
IEEE	Institute of Electrical and Electronics Engineers
NVD	National Vulnerability Database
RBAC	Role-Based Access Control
SRS	Software Requirements Specification
TLS/SSL	Transport Layer Security / Secure Sockets Layer

1.5 References

- IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.
- RFC 793: *Transmission Control Protocol (TCP) Specification*.
- OWASP Secure Headers Project Guidelines.
- Python 3.10+ Documentation ([socket](#), [ssl](#), [asyncio](#)).

2. Overall Description

2.1 Product Perspective

AegisX is a self-contained, lightweight Python platform operating as a client-side network utility with an integrated dashboard UI powered by Streamlit. It functions independently of commercial enterprise vulnerability scanners (e.g., Nessus, Qualys) while standardizing data formatting from native protocol sockets and web endpoints.

2.2 Product Functions

AegisX provides the following high-level features:

- **Multi-Threaded Asynchronous Port Scanning:** Rapid detection of common open TCP ports.
- **Service & Banner Extraction:** Protocol handshake parsing to detect service details.
- **HTTP/HTTPS Security Audit:** Verification of key security response headers.
- **TLS Certificate Inspection:** Certificate validity, trusted chain, and expiration verification.
- **Domain Reconnaissance:** Automated DNS record lookup and WHOIS registry parsing.
- **Calculated Risk Engine:** Deterministic rule-based scoring outputting \$0.0 - 10.0\$ risk ratings.
- **Reporting Engine:** Generation of structured PDF executive reports and CSV data dumps.

2.3 User Classes and Characteristics

User Class	Technical Expertise	Primary Use Case
DevSecOps / Developers	High	Running fast pre-deployment web application posture checks.
Security Analysts	Advanced	Conducting initial passive domain and server reconnaissance.

Academic / Recruiters	Intermediate to High	Code inspection, architecture evaluation, and demonstration testing.
------------------------------	----------------------	--

2.4 Operating Environment

- **Operating System:** Cross-platform (Linux Ubuntu 20.04+, macOS 12+, Windows 10/11).
- **Runtime Environment:** Python 3.10 or higher.
- **Display UI:** Modern Web Browser (Chrome 100+, Firefox 100+, Edge 100+).

2.5 Design and Implementation Constraints

- **Probing Limitations:** Scans are intentionally restricted to TCP connect calls to prevent unprivileged execution failures.
- **No Exploitation:** Active vulnerability exploitation and payloads are explicitly out of scope.
- **Execution Boundary:** Standard scan runtime must complete within 30 seconds over a broadband connection.

2.6 Assumptions and Dependencies

- The target IP/domain is accessible via standard outbound network routing.
- Local host execution environment has active network socket binding privileges.
- Python external libraries ([streamlit](#), [dnspython](#), [reportlab](#), [plotly](#), [requests](#), [pandas](#), [python-whois](#)) are installed via [pip](#).

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Network Port Scanning Engine

- **FR-PORT-01:** The system shall accept valid IPv4, IPv6, or domain name formats as target inputs.
- **FR-PORT-02:** The system shall perform asynchronous TCP socket connections across target ports (defaulting to Top 100 well-known ports).
- **FR-PORT-03:** The system shall enforce a configurable socket timeout parameter (default $\leq 1.5 \text{ seconds}$) per connection attempt to handle unreachable hosts.

3.1.2 Service Detection & Banner Grabbing

- **FR-SERV-01:** The system shall read initial server greeting banners on established TCP ports.
- **FR-SERV-02:** The system shall map identified port and banner combinations against standard service signatures (e.g., HTTP, SSH, FTP, SMTP, MySQL).

3.1.3 HTTP/HTTPS Web Security Auditor

- **FR-WEB-01:** The system shall inspect HTTP response headers for presence and correct configuration of:
 - [Strict-Transport-Security](#) (HSTS)

- Content-Security-Policy (CSP)
 - X-Frame-Options
 - X-Content-Type-Options
 - Referrer-Policy
 - Permissions-Policy
- **FR-WEB-02:** The system shall identify insecure disclosures (e.g., explicit server version information in **Server** or **X-Powered-By** headers).

3.1.4 SSL/TLS Certificate Inspector

- **FR-SSL-01:** The system shall inspect target SSL/TLS certificates over port 443.
- **FR-SSL-02:** The system shall extract issuer information, protocol versions (TLS 1.2, TLS 1.3), signature algorithms, and expiration timestamps.
- **FR-SSL-03:** The system shall flag certificates expiring within 30 days, expired certificates, or self-signed certificates.

3.1.5 DNS & WHOIS Reconnaissance

- **FR-REC-01:** The system shall resolve standard DNS records (**A**, **AAAA**, **MX**, **TXT**, **NS**) for target domains.
- **FR-REC-02:** The system shall retrieve target domain registrar names, registration dates, and expiry dates via WHOIS queries.

3.1.6 Risk Scoring & Recommendation Engine

- **FR-RISK-01:** The system shall calculate an aggregate security score (\$0.0 - 10.0\$) using a weighted rule algorithm based on missing security headers, open sensitive ports, and SSL issues:

$$\text{Risk Score} = \min(10.0, \sum \text{Severity Weights})$$
- **FR-RISK-02:** The system shall pair each detected finding with an actionable remediation guide.

3.1.7 UI & Reporting

- **FR-UI-01:** The system shall display scan progress, metric summaries, data tables, and risk visualizations in a Streamlit web interface.
- **FR-REP-01:** The system shall generate downloadable executive PDF reports formatted via ReportLab and CSV raw dataset exports.

3.2 External Interface Requirements

3.2.1 User Interfaces

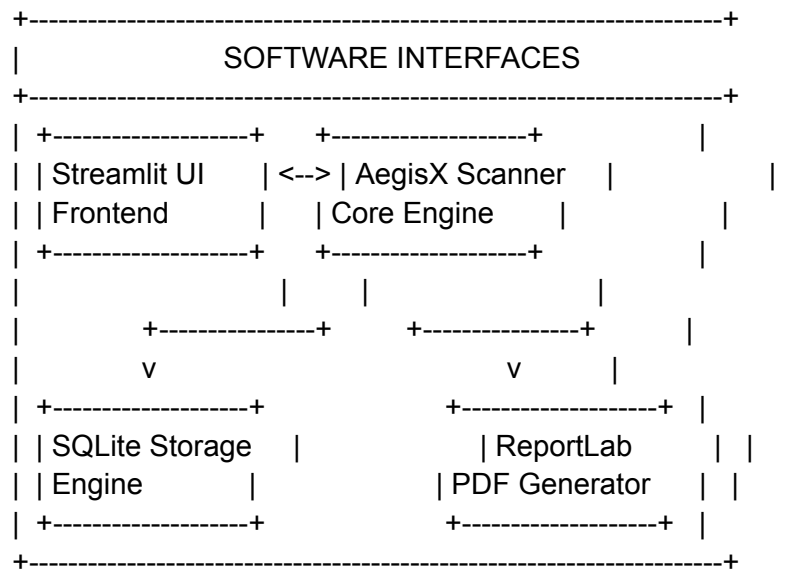
The user interface shall be delivered via Streamlit running locally. It must include:

- Target input bar with domain/IP validation checks.
- Actionable scan controls ("Start Assessment", "Cancel").
- Tabbed result views: Overview, Network Ports, Web Security, SSL Audit, Reconnaissance, Recommendations.
- PDF and CSV export buttons.

3.2.2 Hardware Interfaces

No specialized hardware is required. The system executes on standard x86-64 or ARM64 workstation architecture.

3.2.3 Software Interfaces



Interface	Library / Technology	Purpose
UI Framework	streamlit	Rendering web interface
Database	sqlite3	Storing scan logs and persistent configurations
Network Sockets	socket, ssl	TCP port scanning and certificate retrieval
HTTP Requests	requests	Fetching web headers and checking endpoints
DNS Querying	dnspython	Resolving domain records
WHOIS Lookup	python-whois	Obtaining domain registration details

Visualizations	plotly	Rendering charts and risk gauge displays
PDF Generation	reportlab	Compiling PDF audit reports

3.2.4 Communication Interfaces

The platform uses standard OS socket mechanisms over outbound network interfaces:

- TCP ports 80, 443 for Web/SSL checks.
- TCP connection probing over designated scan ports.
- UDP/TCP Port 53 for DNS resolution queries.
- TCP Port 43 for WHOIS queries.

3.3 Non-Functional Requirements

3.3.1 Performance Requirements

- **NFR-PERF-01:** Standard top-port scans and web audits must complete in ≤ 30 seconds on broadband network connections.
- **NFR-PERF-02:** PDF generation process must complete within 3 seconds post-scan.

3.3.2 Security Requirements

- **NFR-SEC-01 (Input Validation):** All user input strings must undergo strict regular expression checks to prevent command injection risks.
- **NFR-SEC-02 (Memory Safety):** Raw network responses must be sanitized before rendering in the HTML/Streamlit interface to prevent Cross-Site Scripting (XSS).
- **NFR-SEC-03 (Defensive Operations):** Port scanning must be non-intrusive and limited to TCP connect handshakes without sending exploit payloads.

3.3.3 Database Requirements

- **NFR-DB-01:** The persistent storage engine shall use SQLite.
- **NFR-DB-02:** Database schema must support target record storage, scan result metadata, and generated vulnerability findings.

SQL

-- SQLite Core Schema Specification

```
CREATE TABLE IF NOT EXISTS scans (
    scan_id TEXT PRIMARY KEY,
    target TEXT NOT NULL,
    scan_timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    risk_score REAL NOT NULL,
    status TEXT NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS findings (
    finding_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

scan_id TEXT,
category TEXT NOT NULL,
severity TEXT NOT NULL,
title TEXT NOT NULL,
description TEXT,
remediation TEXT,
FOREIGN KEY(scan_id) REFERENCES scans(scan_id)
);

```

3.3.4 Error Handling & Logging

- **NFR-ERR-01:** Unhandled socket timeouts or DNS resolution errors must fail gracefully without crashing the UI thread.
- **NFR-LOG-01:** Scanner logs must be written to structured log files ([aegisx.log](#)) using standard Python [logging](#) modules, categorized by severity levels ([INFO](#), [WARNING](#), [ERROR](#)).

3.3.5 Software Quality Attributes

- **Portability:** Runs on any Python 3.10+ installation with cross-platform OS compatibility.
- **Maintainability:** Code organized into modular packages ([core/](#), [ui/](#), [reports/](#)) following PEP 8 conventions.
- **Usability:** Single-page user dashboard interface requiring zero command-line interaction during assessment execution.

4. Testing & Deployment Requirements

4.1 Testing Requirements

- **Unit Testing:** Coverage for input validation modules, risk scoring logic, and parser modules using [pytest](#).
- **Integration Testing:** Verification of complete scan runs against local test containers (e.g., OWASP Juice Shop, Damn Vulnerable Web Application).
- **Robustness Testing:** Handled execution tests using unreachable targets, silent IP addresses, and malformed domain inputs.

4.2 Deployment Requirements

- **Installation:** Simple step execution:
- Bash

```

git clone https://github.com/placeholder/AegisX.git
cd AegisX
pip install -r requirements.txt
streamlit run app.py

```

-
-
- **Packaging:** Include clean [requirements.txt](#) and modular directory layout for deployment on local workstations or Streamlit Cloud platforms.

5. Future Roadmap & Enhancements

+-----+	
	FUTURE DEVELOPMENT ROADMAP
	-----+
	Phase 2: Intelligence & Persistence
	├── LLM Security Explanations (OpenAI / Gemini API)
	├── NIST NVD CVE Lookup Module Integration
	└── Full Scan History & Trend Tracking (PostgreSQL/SQLite)
	Phase 3: Platform Maturity
	├── Threat Intelligence APIs (AbuseIPDB, VirusTotal, Shodan)
	├── Automated Background Cron Scan Scheduling
	└── Multi-User Authentication & Role-Based Access Control
	-----+

- **AI Context Explanation:** Integrating LLM API calls (OpenAI/Gemini) to provide context-aware risk remediation explanations.
- **CVE Mapping:** Automatic correlation of grabbed service banners against NIST National Vulnerability Database (NVD) records.
- **Threat Intelligence Integration:** Direct API queries against AbuseIPDB, VirusTotal, and Shodan.
- **Scheduled Scanning & RBAC:** Background worker scheduled audits with email alerts and multi-user authentication.